

Day06-Bash脚本定时任务与性能优化

课程目标

- 编写简单的 Bash 脚本
- 掌握循环和条件判断
- 理解正则表达式和 grep
- 配置定时任务(cron)
- 掌握文件归档和传输
- 理解系统性能调优基础

上午 - Bash脚本与定时任务

课前故事：Shell 脚本的力量

为什么需要 Shell 脚本？

没有 shell 脚本的日子

场景：每天备份 10 个服务器的重要文件

手动操作（耗时且容易出错）：

- SSH server1
 cd /data
 tar -czf backup.tar.gz important/
- SSH server2 ... 重复上述操作 ...
- ...

问题：每天重复、容易遗漏、无法记录

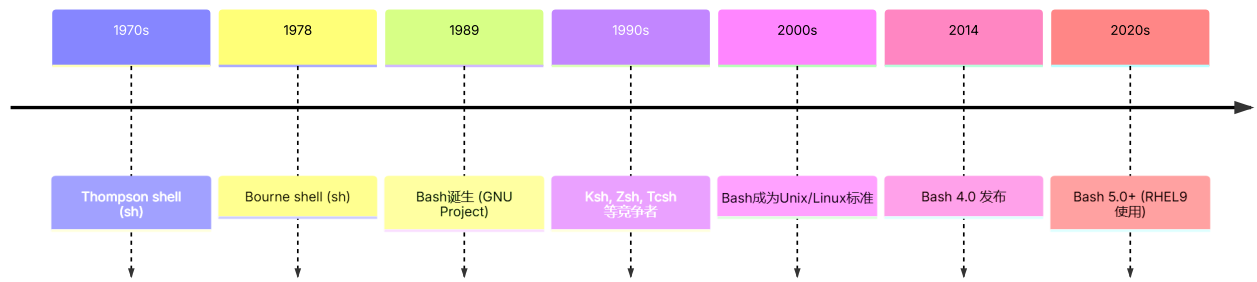
解决方案：Shell 脚本

```
#!/bin/bash
for server in server{1..10}; do
  ssh $server "backup.sh"
done
```

一行命令完成所有工作！

Shell 的发展历史：

Shell 脚本发展史



什么是 Bash?

Bash = Bourne Again Shell

特性:

- Brian Fox 为 GNU Project 开发
- 兼容 Bourne shell (sh)
- 命令行解释器和脚本语言
- Linux 系统的默认 shell
- RHEL9 使用 Bash 5.1

为什么选择 Bash?

- 广泛支持: 几乎所有 Linux 发行版
- 功能强大: 脚本语言完整
- 易学易用: 语法直观
- 社区活跃: 大量现成脚本

第一部分: Bash脚本基础

为什么需要学习Bash脚本?

通过课前故事, 我们了解了Shell脚本的发展历史。现在让我们开始实际编写脚本。

本部分将学习:

- 脚本的创建和执行
- 变量的使用和输出
- 引号和转义字符

这些是编写任何Bash脚本的基础。

1.1 脚本创建和执行

什么是 **Shell** 脚本：

Shell 脚本是包含 Linux 命令的文本文件

- 自动化重复任务
- 提高工作效率
- 系统管理必备技能
- 可重复执行、可维护、可记录

脚本第一行：**she-bang**

编写脚本时，第一行总是以 she-bang 开始。这是脚本最关键的组成部分。

为什么 **she-bang** 如此重要？

- 它告诉系统用哪个程序来解释这个脚本
- 没有它，脚本无法直接执行
- 它是脚本与系统之间的"语言声明"

```
#!/bin/bash
```

SHELL

说明：

- **#!** 称为 she-bang 或 hashbang
- 指定脚本解释器
- 必须是脚本的第一行
- 告诉系统用什么程序来执行这个脚本

为什么需要 **she-bang**？

没有 she-bang 的脚本:

```
vim myscript.sh
echo "Hello"
```

运行时:

```
bash myscript.sh    # 需要明确指定解释器
```

有 she-bang 的脚本:

```
vim myscript.sh
#!/bin/bash
echo "Hello"
```

运行时:

```
./myscript.sh    # 系统自动使用 she-bang 指定的解释器
```

⚠ 易错点: she-bang 格式

☒ 正确格式:

```
#!/bin/bash
```

注意: #! 和路径之间没有空格

X 常见错误:

```
# !/bin/bash    # 缺少 #
```

```
# # /bin/bash   # 多了空格
```

```
# #!/usr/bin/bash # 路径错误
```

检查解释器路径:

```
which bash
```

```
# /usr/bin/bash
```

RHEL9 中 bash 的位置:

```
ls -l /bin/bash
```

```
# /bin/bash -> /usr/bin/bash
```

创建可执行脚本:

步骤1: 创建脚本文件

```
vim ~/bin/hello
```

```
#!/bin/bash
```

这是一个简单的脚本

```
echo 'Hello, World'
```

```
echo "当前用户: $USER"
```

```
echo "当前目录: $PWD"
```

```
echo "主机名: $HOSTNAME"
```

步骤2: 添加执行权限

```
chmod +x ~/bin/hello
```

步骤3: 验证权限

```
ls -l ~/bin/hello
```

```
# -rwxr-xr-x. 1 student student ... hello
```

```
#           ↑
```

```
#           x 表示可执行
```

步骤4: 运行脚本

```
~/bin/hello
```

输出:

```
# Hello, World
```

```
# 当前用户: student
```

```
# 当前目录: /home/student
```

```
# 主机名: servera
```

⚠ 技巧: 脚本位置

实际工作中脚本通常放在当前目录:

```
vim myscript.sh
chmod +x myscript.sh
./myscript.sh
```

实际工作中也可能放在 ~/bin/:

```
mkdir -p ~/bin
vim ~/bin/myscript.sh
chmod +x ~/bin/myscript.sh
```

执行方式:

1. 绝对路径

```
~/bin/myscript.sh
/home/student/bin/myscript.sh
```

2. 相对路径

```
./myscript.sh          # 当前目录下的脚本
```

3. 如果脚本所在执行路径 ~/bin 在 \$PATH 中, 可直接用文件名

检查 PATH:

```
echo $PATH
```

或将脚本所在执行路径 ~/bin 添加到 PATH (例如在 ~/.bashrc 中) :

```
echo 'export PATH="$HOME/bin:$PATH"' >> ~/.bashrc
source ~/.bashrc
```

然后可以直接运行:

```
myscript.sh
```

关键点:

- # 1. 确保脚本有执行权限 (chmod +x)
- # 2. 使用正确的路径运行脚本
- # 3. 执行路径 在 PATH 中可直接用文件名执行

运行脚本的两种方式:

```
# 方式1: 作为可执行文件运行
/path-to/myscript.sh

# 前提条件:
# - 脚本有执行权限 (chmod +x)以及读取权限
# - 脚本有正确的 she-bang

# 方式2: 显式指定解释器
bash /path-to/myscript.sh

# 优点:
# - 不需要执行权限, 但需要读取权限
# - 不需要 she-bang
# - 可以用不同的解释器测试
```

脚本权限详解:

```
# 脚本权限说明:
chmod +x script.sh      # 添加执行权限
chmod 755 script.sh     # rwxr-xr-x
chmod 700 script.sh     # rwx-----

# 权限选择:
# 755 (rwxr-xr-x) - 所有者可执行, 其他人可读可执行
# 744 (rwxr--r--) - 所有者可执行, 其他人只读
# 700 (rwx-----) - 只有所有者可以访问

# 安全考虑:
# - 包含敏感信息的脚本应该用 700
# - 公共脚本可以用 755
# - 不应该给脚本写权限 (666) 或执行权限给所有人 (777)
```

1.2 变量和输出

掌握了脚本的创建和执行后, 接下来我们需要让脚本"活"起来。

为什么需要变量?

- 存储用户输入、配置信息

- 避免重复输入相同内容
- 让脚本可重用、可维护

使用变量：

```
#!/bin/bash
# 定义变量
name="World"
echo "Hello, $name"

# 变量命名规则：
name="Student"      # ✓ 正确
user_name="Alice"   # ✓ 正确（下划线）
2name="Bad"         # ✗ 错误（不能以数字开头）
user-name="Bad"     # ✗ 错误（不能使用连字符）
```

SHELL

变量使用规则：

- 赋值时：变量名不加 `$` 符号
- 使用时：变量名加 `$` 符号
- 建议用双引号包围变量引用

⚠ 易错点：变量使用

```
# ✓ 正确用法：
name="World"
echo "Hello, $name"      # 输出：Hello, World

# ✗ 错误用法：
name="World"
echo 'Hello, $name'      # 输出：Hello, $name（单引号不展开变量）
echo "Hello, ${name}"    # ✓ 可以用 ${} 明确变量边界

# 场景示例：
filename="report.txt"
echo "File: $filename"    # File: report.txt
echo "File: ${filename}.old" # File: report.txt.old
echo 'File: $filename.old' # File: $filename.old（不展开）
```

SHELL

常用内置变量：


```
#!/bin/bash
# 用户相关变量
echo "当前用户: $USER"
echo "用户ID: $(id -u)"
echo "主目录: $HOME"

# 系统相关变量
echo "主机名: $HOSTNAME"
echo "当前路径: $PWD"
echo "Shell: $SHELL"

# 脚本相关变量
echo "脚本名称: $0"
echo "参数数量: $# "
echo "所有参数: $@"
echo "第一个参数: $1"
echo "第二个参数: $2"
```

输出重定向：

脚本运行时会产生大量输出信息。除了显示在屏幕上，我们经常需要将输出保存到文件中。这就是输出重定向的作用。

```
#!/bin/bash
# 标准输出重定向到文件
echo "正常输出" > /tmp/output.log
echo "追加输出" >> /tmp/output.log

# 标准错误重定向
echo "错误信息" >&2

# 两者都重定向
echo "消息" > /tmp/log.txt 2>&1
echo "消息" &> /tmp/log.txt

# 解释:
# > file      # 将标准输出重定向到文件 (覆盖)
# >> file     # 将标准输出追加到文件
# 2> file     # 将标准错误重定向到文件
# > file 2>&1  # 将标准错误重定向到标准输出
# &> file     # bash 简写, 等价于 > file 2>&1

# 实际应用:
echo "备份完成" > /var/log/backup.log
echo "备份失败" >&2
```

Bash 脚本代码风格指南:

良好的代码风格让脚本更易读、易维护、易调试。

最佳实践	说明	示例
变量命名	使用小写+下划线	my_var, user_count
函数命名	使用小写+下划线	backup_files(), check_status()
常量命名	使用大写+下划线	MAX_RETRIES=5, LOG_DIR="/var/log"
注释	解释"为什么"而非"是什么"	# 检查文件是否存在, 避免覆盖
缩进	使用4空格, 保持一致性	避免 Tab 混用空格

命名对比:

```
# ✔ 推荐的命名风格
my_backup_dir="/data/backup"
MAX_RETRIES=3
backup_files() {
    ...
}

# ✗ 不推荐的命名风格
MyBackupDir="/data/backup"      # 驼峰命名不推荐
max-retries=3                   # 不使用连字符
backupFiles() {                 # 驼峰命名不推荐
    ...
}
```

为什么遵循代码风格？

- RHCSA考试中脚本可读性影响评分
- 生产环境代码需要多人协作维护
- 清晰的命名减少bug和调试时间

1.3 引号和转义

在Bash脚本中，某些字符（如 `$`、`*`、`?`）有特殊含义。当我们需要使用这些字符的字面值时，就需要进行转义或使用引号。这是初学者最容易出错的地方。

特殊字符处理：

方式	说明	示例
反斜杠 <code>\</code>	转义单个字符	<code>echo \ \$HOME</code>
单引号 <code>' '</code>	完全字面值	<code>echo ' \$HOME '</code>
双引号 <code>" "</code>	保留变量含义	<code>echo " \$HOME "</code>

示例：

```
# 输出 $HOME 字面值
echo '$HOME'

# 输出变量值
echo "$HOME"

# 转义特殊字符
echo \# This is not a comment
```

练习名称	对应章节	说明
练习： Write Simple Bash Scripts	第01章 第01节 P23	编写简单的Bash脚本

第二部分：循环和条件

为什么需要循环和条件？

到目前为止，我们学习的脚本是按顺序执行的。但在实际工作中，我们经常需要：

- 对多个文件执行相同操作（循环）
- 根据不同情况执行不同命令（条件判断）

这两者是脚本自动化的核心能力。

2.1 for 循环

for 循环的应用场景：

- 遍历文件列表
- 批量处理用户
- 重复执行操作

for 循环语法：

```
for variable in list;do  
    command  
done
```

基本示例：

```
# 遍历列表  
for host in host1 host2 host3; do  
    echo $host  
done  
  
# 输出：  
# host1  
# host2  
# host3
```

使用大括号扩展：

```
# 生成序列  
for num in {1..5}; do  
    echo "Number: $num"  
done  
  
# 输出：  
# Number: 1  
# Number: 2  
# Number: 3  
# Number: 4  
# Number: 5
```

更多大括号扩展示例：

```
# 列表扩展
for host in host{1,2,3}; do
    echo $host
done
# 输出: host1, host2, host3

# 遍历文件列表
for file in file*.txt; do
    ls -l $file
done

# 序列扩展 (与 seq 命令相同)
for num in $(seq 2 2 10); do
    echo "$num"
done
# 输出: 2, 4, 6, 8, 10

# 获取内核安装包日期
for package in $(rpm -qa | grep kernel); do
    echo "$package was installed on $(date -d @$(rpm -q --qf "%{INSTALLTIME}" $package))"
done
```

遍历文件:

```
# 遍历匹配的文件
for file in file*; do
    ls -l $file
done
```

2.2 退出代码 (Exit Codes)

在编写循环和条件脚本之前，我们需要理解一个重要概念：退出代码。

为什么需要退出代码？

- 判断命令是否成功执行
- 在脚本中做出正确的决策

- 调试脚本错误

退出代码详解：

SHELL

```
# exit 命令用于退出脚本
# 范围：0-255
# 0 = 成功, 非零 = 错误

# 示例：检查命令退出状态
systemctl is-active httpd > /dev/null 2>&1
if [ $? -ne 0 ]; then
    echo "httpd is not running"
fi

# $? 变量存储上一条命令的退出代码
# 注意：每次执行命令后，$? 会被更新
```

退出代码含义：

代码	含义
0	成功
1	一般错误
2	误用 shell 命令
126	命令无法执行
127	命令未找到
128	无效的退出参数
130	被 Ctrl+C 中断
255	退出状态越界

SHELL

```
# 示例：脚本中使用 exit
#!/bin/bash
echo "Hello, World"
exit 0

# 检查退出代码
./script.sh
echo $? # 输出：0
```

2.3 条件判断

学习了退出代码后，我们现在可以使用它来做决策了。条件判断让脚本能够根据不同情况执行不同操作。

条件判断与退出代码的关系：

- 退出代码是判断的依据
- `$?` 获取上条命令的退出代码
- 条件判断根据退出代码决定执行路径

if/then 结构：

```
if condition; then
    command
fi
```

SHELL

示例：

```
# 检查服务状态
systemctl is-active httpd > /dev/null 2>&1
if [ $? -ne 0 ]; then
    systemctl start httpd
fi
```

SHELL

if/then/else 结构：

```
if condition; then
    command1
else
    command2
fi
```

SHELL

示例：


```
systemctl is-active httpd > /dev/null 2>&1
if [ $? -ne 0 ]; then
    systemctl start httpd
else
    systemctl stop httpd
fi
```

if/then/elif/else 结构:

```
if condition1; then
    command1
elif condition2; then
    command2
else
    command3
fi
```

2.4 测试条件

条件判断需要具体的测试条件。Bash提供了数字、字符串和文件的测试方法。

数字比较:

运算符	说明
<code>-eq</code>	等于
<code>-ne</code>	不等于
<code>-gt</code>	大于
<code>-ge</code>	大于等于
<code>-lt</code>	小于
<code>-le</code>	小于等于

示例:

```
if [ 1 -gt 0 ]; then
    echo "1 is greater than 0"
fi
```

字符串比较：

运算符	说明
=	等于
!=	不等于
-z	空字符串
-n	非空字符串

示例：

```
if [ "$name" = "root" ]; then
    echo "User is root"
fi
```

文件测试：

测试	说明
-f	普通文件
-d	目录
-e	存在
-r	可读
-w	可写
-x	可执行

示例：

```
if [ -f /etc/passwd ]; then
    echo "File exists"
fi
```

练习名称	对应章节	说明
练习：Loops and Conditional Constructs in Scripts	第01章 第02节 P31	脚本中的循环和条件结构

第三部分：正则表达式与grep

为什么需要正则表达式？

在脚本中，我们经常需要：

- 从文件中查找特定模式
- 验证输入格式（如邮箱、IP地址）
- 提取和过滤文本信息

正则表达式是处理这些任务的强大工具。

3.1 正则表达式基础

什么是正则表达式：

正则表达式是用于模式匹配的强大工具

-  匹配任意单个字符
-  匹配前一个字符零次或多次
-  匹配任意字符零次或多次
-  行首
-  行尾

示例：

```
# 匹配以 cat 开头的行
grep '^cat' file.txt

# 匹配以 dog 结尾的行
grep 'dog$' file.txt

# 匹配 cat、c5t、cQt 等
grep 'c.t' file.txt
```

字符类：

模式	说明
<code>[abc]</code>	匹配 a、b 或 c
<code>[a-z]</code>	匹配小写字母
<code>[0-9]</code>	匹配数字
<code>[^abc]</code>	不匹配 a、b、c

示例：

```
# 匹配 cat、cot、cut
grep 'c[aou]t' file.txt

# 匹配数字
grep '[0-9]' file.txt
```

完整正则表达式参考表：

元字符	说明
<code>.</code>	匹配任意单个字符
<code>?</code>	前面的项目是可选的，最多匹配一次
<code>*</code>	前面的项目匹配零次或多次
<code>+</code>	前面的项目匹配一次或多次
<code>{n}</code>	前面的项目恰好匹配 n 次
<code>{n,}</code>	前面的项目匹配 n 次或更多次

元字符	说明
<code>{,m}</code>	前面的项目最多匹配 m 次
<code>{n,m}</code>	前面的项目至少匹配 n 次, 不超过 m 次
<code>^</code>	行首
<code>\$</code>	行尾
<code>[:alnum:]</code>	字母数字字符
<code>[:alpha:]</code>	字母字符
<code>[:digit:]</code>	数字
<code>[:lower:]</code>	小写字母
<code>[:upper:]</code>	大写字母
<code>[:space:]</code>	空格字符
<code>\b</code>	匹配词语两侧的空字符串
<code>\B</code>	匹配词语中间的空字符串
<code>\<</code>	匹配词语开头的空字符串
<code>\></code>	匹配词语末尾的空字符串
<code>\w</code>	匹配词语组分
<code>\W</code>	匹配非词语组分
<code>\s</code>	匹配空格
<code>\S</code>	匹配非空格

正则表达式示例：

```
# 行首匹配
grep '^cat' file.txt

# 行尾匹配
grep 'dog$' file.txt

# 匹配 c 和 t 之间任意一个字符
grep 'c.t' file.txt

# 匹配 c 开头 t 结尾, 中间任意字符
grep 'c.*t' file.txt

# 匹配 c 开头 t 结尾, 中间恰好 2 个字符
grep 'c.{2}t' file.txt

# 匹配整个单词 cat (使用词语边界)
grep '\bcat\b' file.txt
```

3.2 grep 命令

grep 常用选项:

选项	说明
<code>-i</code>	忽略大小写
<code>-v</code>	反向匹配
<code>-r</code>	递归搜索
<code>-n</code>	显示行号
<code>-c</code>	计数
<code>-A n</code>	显示匹配后 n 行
<code>-B n</code>	显示匹配前 n 行

示例:

```
# 忽略大小写搜索
grep -i 'server' /etc/hosts

# 搜索不包含某字符串的行
grep -v '^#' /etc/fstab

# 递归搜索
grep -r 'ERROR' /var/log/
```

grep 与管道：

```
# 在 ps 输出中搜索进程
ps aux | grep httpd

# 搜索并显示行号
grep -n 'root' /etc/passwd
```

练习名称	对应章节	说明
练习：console regex	第01章 第03节 P41	使用正则表达式匹配输出文本

第四部分：定时任务

从脚本到自动化：

现在我们已经能够编写脚本，但是每次手动运行脚本仍然不够自动化。定时任务让系统能够在指定时间自动执行脚本。

定时任务的应用场景：

- 夜间备份
 - 定期日志清理
 - 周期性系统检查
-

4.1 at - 一次性任务

at 命令:

在指定时间运行命令

```
at 14:30
```

进入交互模式

```
at> echo "Hello" > /tmp/hello.txt
```

```
at> Ctrl+D
```

查看待执行任务

```
atq
```

删除任务

```
atrm jobnumber
```

SHELL

时间格式 (TIMESPEC):

格式	说明
now +5min	5分钟后
teatime	下午4点 (16:00)
teatime tomorrow	明天下午4点
noon	中午12点
midnight	午夜 (00:00)
14:30	14:30
02:00pm	下午2点
4pm	下午4点
noon +4 days	4天后的中午
5pm august 3 2021	2021年8月3日下午5点

at 命令详细示例:


```
# 从脚本导入指令
at now +5min < myscript

# 查看任务详细信息
at -c jobnumber

# at 队列说明
# atd 提供 a-z 共 26 个队列
# 按字母排序，越后面的队列优先级越低
```

时间格式参考文件：

```
# 查看完整的时间格式文档
cat /usr/share/doc/at/timespec
```

4.2 cron - 周期性任务

crontab 命令：

命令	说明
<code>crontab -l</code>	列出任务
<code>crontab -e</code>	编辑任务
<code>crontab -r</code>	删除所有任务
<code>crontab -u user</code>	管理用户任务

cron 时间格式：

```
分钟 小时 日期 月份 星期 命令
* * * * * command
```

注意：日期和星期几是 **AND** 的关系（两者都必须满足）

特殊字符：

字符	说明	示例
*	任意值	* * * * *
,	列表分隔符	1,2,3
-	范围	1-5
*/n	每 n 个单位	*/5

cron 时间字段详解：

字段	范围	允许的特殊字符
分钟	0-59	* , - /
小时	0-23	* , - /
日期	1-31	* , - /
月份	1-12	* , - /
星期	0-7 (0或7是周日)	* , - /

cron 示例详解：

```
# 每天凌晨2点运行
0 2 * * * /root/backup.sh

# 每小时运行
0 * * * * /root/check.sh

# 每周一早上9点
0 9 * * 1 /root/weekly.sh

# 每5分钟运行一次
*/5 * * * * /root/frequent.sh

# 工作日(周一到周五)每小时
0 9-17 * * 1-5 /root/workday.sh

# 每年2月2日上午9点
0 9 2 2 * /usr/local/bin/yearly_backup

# 七月每周五的上午9点和下午5点
*/5 9,17 * 7 5 echo "Chime" | mail -s "Chime" user

# 每个工作日上午9点发送邮件
0 9 * * 1-5 mutt -s 'Checking in' user@example.com

# 特殊时间格式示例
teatime tomorrow      # 明天下午4点
noon +4 days           # 4天后的中午
5pm august 3 2021      # 2021年8月3日下午5点
```

练习名称	对应章节	说明
练习：SCHEDULING RECURRING SYSTEM JOBS	第02章 第02节 P60	配置周期性系统任务 (cron)

下午 - 文件归档与性能优化

下午课程安排：

上午我们学习了脚本和定时任务。下午我们将专注于：

- 1. 文件归档与传输 - 如何高效备份和传输数据
- 2. 系统性能调优 - 如何让系统运行更高效
- 3. 时间同步 - 确保系统时间准确

这些是系统管理员日常维护的核心技能。

第五部分：文件归档与传输

为什么需要文件归档？

在定时任务中，最常见的操作之一是备份。文件归档和传输工具让备份工作变得高效可靠。

本部分将学习：

- tar: 创建和管理归档文件
- scp/sftp: 安全传输文件
- rsync: 高效同步文件

5.1 tar 命令

tar 是什么？

tar (Tape Archive) 是Linux中最常用的归档工具。它可以将多个文件打包成一个文件，并支持压缩。

为什么使用 tar？

- 将多个文件打包成一个
- 保留文件权限和属性
- 支持多种压缩格式

tar 基本选项：

选项	说明
-c	创建归档

选项	说明
-x	提取归档
-t	列出内容
-v	显示详细信息
-f	指定文件名

创建归档:

SHELL

```
# 创建基本归档
tar -cf archive.tar file1 file2 dir1

# 创建压缩归档 (gzip)
tar -czf archive.tar.gz /etc/hosts

# 创建压缩归档 (bzip2)
tar -cjf archive.tar.bz2 /etc/hosts

# 创建压缩归档 (xz)
tar -cJf archive.tar.xz /etc/hosts
```

参数说明:

- -z = gzip 压缩
- -j = bzip2 压缩
- -J = xz 压缩

提取归档:

SHELL

```
# 提取 gzip 归档
tar -xzf archive.tar.gz

# 提取到指定目录
tar -xzf archive.tar.gz -C /tmp/

# 保留权限提取
tar -xpf archive.tar
```

查看归档内容:

SHELL

```
# 列出归档内容
tar -tzf archive.tar.gz

# 详细列出
tar -tvf archive.tar.gz
```

归档文件路径也会原封不动的保存下来。

路径保存说明:

路径类型	打包示例	解压行为
绝对路径 (如 /etc/hosts)	<code>tar -czf a.tgz /etc/hosts</code>	恢复到 <code>/etc/...</code> (⚠️ 覆盖原文件!)
相对路径 (如 ./etc/hosts)	<code>tar -czf a.tgz ./etc/hosts</code> 或 <code>cd /etc && tar -czf a.tgz *</code>	恢复到当前目录 <code>./etc/hosts</code>

最佳实践建议:

- 推荐使用相对路径打包，便于灵活控制解压位置
- 避免使用绝对路径打包，可能覆盖系统重要文件

解压时的路径处理:

SHELL

```
# 查看归档中的路径 (注意是否为绝对路径)
tar -tzf archive.tar.gz

# 解压到指定目录 (推荐)
tar -xzf archive.tar.gz -C /target/dir/

# 解压时剥离路径前缀 (--strip-components)
tar -xzf archive.tar.gz -C /target/dir/ --strip-components=1
```

⚠️ tar 安全警告 (基于 CVE 漏洞分析)

tar 归档文件可能被恶意构造，存在以下安全风险:

安全风险	说明	防护措施
路径遍历攻击	恶意归档使用 <code>../</code> 逃脱目标目录，覆盖任意文件	始终使用 <code>-C</code> 指定空目录
符号链接攻击	符号链接指向敏感系统位置 (如 <code>/etc/shadow</code>)	谨慎处理不可信归档
绝对路径覆盖	使用绝对路径直接覆盖系统关键文件	避免使用 <code>-P</code> 标志

GNU tar 的默认安全机制：

- GNU tar 会自动去除绝对路径的前导斜杠 `/`
- 解压时显示警告：`Removing leading '/' from member names`
- 这是内置的安全防护，但不能完全依赖它

安全检查清单：

SHELL

```
# 1. 查看归档内容 (检查可疑路径和符号链接)
tar -tzf archive.tar.gz

# 2. 解压到隔离的空目录
mkdir -p /tmp/extract
tar -xzf archive.tar.gz -C /tmp/extract/

# 3. 验证路径未逃脱目标目录
ls -la /tmp/extract/

# 4. 确认无误后再移动到目标位置
mv /tmp/extract/* /final/destination/
```

RHCSA 考试注意事项：

- 考试环境中的归档通常是安全的
- 但养成良好安全习惯很重要
- 始终使用 `-C` 指定解压目录
- 避免使用 `-P` (保留绝对路径) 标志

参考资料：[GNU tar Security](#) | [ZIP Slip 漏洞](#)

5.2 scp 和 sftp

远程文件传输：

归档文件通常需要传输到远程服务器备份。Linux提供了多种安全传输方式。

⚠ **红帽安全建议 (2025更新)**： scp 协议存在安全隐患，可能受路径遍历攻击影响。Red Hat 建议使用 **rsync** 或 **sftp** 替代 scp 进行文件传输。

scp - 安全复制：

```
# 复制文件到远程
scp file.txt user@remote:/path/

# 从远程复制
scp user@remote:/path/file.txt /local/path

# 复制目录
scp -r /local/dir user@remote:/remote/path
```

SHELL

sftp - 安全FTP：

```
# 建立 sftp 连接
sftp user@remote

# sftp 交互命令
sftp> ls          # 列出远程文件
sftp> pwd         # 远程当前目录
sftp> lpwd        # 本地当前目录
sftp> put file.txt # 上传文件
sftp> get file.txt # 下载文件
sftp> mkdir dir   # 创建目录
sftp> exit        # 退出
```

SHELL

5.3 rsync 同步

为什么推荐 **rsync**？

rsync 是比 scp 更现代、更高效的同步工具。特别适合：

- 大文件传输
- 重复同步
- 增量备份

rsync 命令：

SHELL

```
# 同步本地目录
rsync -av /source/ /dest/

# 同步到远程
rsync -av /local/ user@remote:/remote/

# 同步远程到本地
rsync -av user@remote:/remote/ /local/
```

常用选项：

选项	说明
-a	归档模式，保留所有属性
-v	显示详细信息
-z	压缩传输
-n	模拟运行
--delete	删除目标中多余的文件

rsync -a 包含的选项：

- **-r** 递归
- **-l** 保留符号链接
- **-p** 保留权限
- **-t** 保留时间戳
- **-g** 保留组
- **-o** 保留所有者
- **-D** 保留设备文件
- **-H** 保留硬链接
- **-A** 保留 ACL
- **-X** 保留 SELinux

rsync 详细示例：

SHELL

```
# 同步本地目录（增量拷贝，仅复制差异部分）
rsync -av /source/ /dest/

# 同步到远程
rsync -av /local/ user@remote:/remote/

# 同步远程到本地
rsync -av user@remote:/remote/ /local/

# 模拟运行（显示需要同步的文件，不真正同步）
rsync -avn /source/ /dest/

# 删除目标中多余的文件
rsync -av --delete /source/ /dest/

# 安全同步选项（推荐用于生产环境）
rsync -avz --delete --checksum /source/ user@remote:/dest/
```

rsync 安全选项说明：

选项	安全作用
--checksum	基于文件内容校验，而非时间/大小
-z	压缩传输，减少网络暴露
--delete	删除目标多余文件（谨慎使用）
-n	模拟运行，预览变更

rsync vs scp：

特性	rsync	scp
传输方式	增量拷贝	全量拷贝
效率	高 (仅传输差异)	低 (传输所有内容)
适用场景	大量文件/重复同步	小文件/一次性传输
安全性	基于 SSH	基于 SSH

⚠ 注意：红帽建议不要再使用 scp，推荐使用 rsync 或 sftp

练习名称	对应章节	说明
练习：ARCHIVING AND TRANSFERRING FILES	第04章	归档和传输文件 (tar, scp, rsync)

第六部分：系统性能调优

为什么需要性能调优？

完成了文件备份和传输后，我们需要确保系统高效运行。RHEL9提供了多种性能优化工具。

本部分将学习：

- tuned: 预设性能配置
 - nice/renice: 进程优先级管理
-

6.1 tuned 配置文件

tuned 简介：

tuned 是系统性能调优守护进程

- 静态调优：应用预设配置
- 动态调优：根据负载调整

查看当前配置：

```
# 查看当前激活的配置
tuned-adm active

# 列出所有可用配置
tuned-adm list

# 获取推荐配置
tuned-adm recommend
```

常用 tuned 配置文件对比：

配置文件	适用场景	特点
virtual-guest	虚拟机	降低虚拟化开销，适合大多数虚拟环境
throughput-performance	高吞吐量服务器	优化磁盘/网络IO，适合文件服务器
latency-performance	低延迟应用	数据库/交易系统，降低响应延迟
powersave	笔记本电脑	省电优先，降低CPU频率
balanced	通用场景	平衡性能和功耗
virtual-host	KVM宿主机	虚拟化宿主机优化

查看配置详情：

```
# 查看推荐配置的依据
tuned-adm recommend
# 输出示例：virtual-guest

# 查看当前生效的配置
tuned-adm active

# 查看指定配置的详细信息
tuned-adm profile_info virtual-guest
```

切换配置文件：

```
# 应用指定配置
tuned-adm profile throughput-performance

# 验证
tuned-adm active

# 禁用 tuned
tuned-adm off
```

6.2 nice 和进程优先级

从全局到局部：

tuned 是全局性的系统调优，而 nice/renice 是针对单个进程的优先级调整。

nice 的应用场景：

- 后台备份任务降低优先级
- 关键服务提高优先级
- 避免重要任务被占用资源

nice 值：

- 范围：-20(最高优先级) 到 19(最低优先级)
- 默认值：0
- nice 值越高，优先级越低

使用 **nice** 启动进程：

```
# 使用默认 nice 值(10)启动
nice command

# 指定 nice 值启动
nice -n 15 command

# 示例
nice -n 19 tar -cf archive.tar /large/directory
```

修改进程 **nice** 值:

SHELL

```
# 使用 renice 修改
renice -n 19 -p 1234

# 在 top 中修改
top
# 按 r 键
# 输入 PID
# 输入新的 nice 值
```

注意:

- 只有 root 可以降低 nice 值(提高优先级)
- 普通用户只能提高自己的进程 nice 值

练习名称	对应章节	说明
练习: ADJUSTING TUNING PROFILES	第05章 第01节	调整tuned配置文件

第七部分：时间同步

为什么时间同步很重要？

在结束今天的课程前，我们还需要确保系统时间准确。这对于：

- 定时任务的正确执行
- 日志时间戳的一致性
- 分布式系统的协调

都是至关重要的。

7.1 chronyd 时间同步

查看时间状态:

查看时间信息

```
timedatectl
```

输出示例:

```
# Local time: Wed 2025-01-15 10:00:00 CST
```

```
# Universal time: Wed 2025-01-15 02:00:00 UTC
```

```
# RTC time: Wed 2025-01-15 02:00:00
```

```
# Time zone: Asia/Shanghai (CST, +0800)
```

```
# System clock synchronized: yes
```

```
# NTP service: active
```

查看 NTP 同步源:

查看时间同步源

```
chronyc sources -v
```

输出说明:

```
# ^* = 已同步且当前最佳
```

```
# ^+ = 已同步但非最佳
```

```
# ^? = 不可用
```

配置 NTP 服务器:

编辑配置文件

```
vim /etc/chrony.conf
```

添加或修改服务器

```
server classroom.example.com iburst
```

重启服务

```
systemctl restart chronyd
```

```
systemctl enable chronyd
```

设置时区:

```
# 列出时区
timedatectl list-timezones

# 设置时区
timedatectl set-timezone Asia/Shanghai

# 验证
timedatectl
```

chrony vs ntpd 优势对比:

特性	chrony	ntpd
间断网络	✔ 快速恢复同步	✘ 同步较慢
虚拟机环境	✔ 适应时钟漂移	✘ 精度下降明显
温度变化	✔ 自动补偿	✘ 时钟误差增大
局域网精度	微秒级	毫秒级
互联网精度	毫秒级	秒级

为什么 RHEL9 使用 chrony?

chrony 在现代环境中表现更好:

- 虚拟机时钟经常不稳定, chrony 能快速适应
- 间断网络连接时恢复更快
- 对温度变化更鲁棒 (物理时钟受温度影响)

chronyc 关键命令详解:


```
# 查看同步状态（最常用）
chronyc tracking

# 输出关键信息：
# - Reference ID: 当前同步的服务器
# - Stratum: 层级（越小越好）
# - System time: 与NTP时间的偏差
# - Last offset: 最后一次的时间偏差

# 查看时间源
chronyc sources

# 输出符号说明：
# ^* = 当前正在同步的源（最佳）
# ^+ = 候选同步源
# ^? = 不可达的源
# ^- = 被排除的源

# 查看时间源统计信息
chronyc sourcestats

# 显示每个源的详细统计

# 手动步进时间（慎用，仅在大偏差时使用）
chronyc makestep
```

/etc/chrony.conf 关键参数：

参数	说明	典型值
<code>server</code>	NTP服务器地址	<code>pool.ntp.org</code>
<code>iburst</code>	加快初始同步（发送8个包）	建议
<code>makestep</code>	大幅度时间调整	<code>makestep 1.0 3</code>
<code>driftfile</code>	频率漂移文件	<code>/var/lib/chrony/drift</code>

要点：

- 记住 `server` 配置语法
 - 记住使用 `iburst` 加快同步
 - 使用 `chronyc sources` 检查同步状态
 - `^*` 表示已同步的时间源
-

练习名称	对应章节	说明
练习：CONFIGURING CHRONY TIME SERVICE	第05章	配置chrony时间同步服务

今日总结

命令清单

脚本与搜索：

```
bash script.sh          # 执行脚本
grep pattern file       # 搜索模式
grep -i pattern file    # 忽略大小写
grep -r pattern dir     # 递归搜索
```

SHELL

定时任务：

```
crontab -e              # 编辑cron
crontab -l              # 列出cron
at time                 # at任务
atq                     # 查看待执行任务
```

SHELL

文件归档：

```
tar -czf archive.tar.gz dir  # 创建gzip归档
tar -xzf archive.tar.gz      # 提取gzip归档
tar -cjf archive.tar.bz2 dir # 创建bzip2归档
tar -xjf archive.tar.bz2     # 提取bzip2归档
```

SHELL

文件传输：

SHELL

```
scp file user@host:/path      # 安全复制
sftp user@host                # 安全FTP
rsync -av src/ dest/          # 同步文件
```

性能调优：

SHELL

```
tuned-adm active              # 查看配置
tuned-adm profile name        # 应用配置
nice -n 10 command            # 设置优先级
renice -n 19 -p PID           # 修改优先级
```

时间同步：

SHELL

```
timedatectl                  # 查看时间
timedatectl set-timezone TZ  # 设置时区
chronyc sources -v           # 查看NTP源
```

常见任务

1. 创建定时任务

SHELL

```
crontab -e
# 添加: */5 * * * * /path/to/script
```

2. 创建压缩归档

SHELL

```
tar -cjf /root/backup.tar.bz2 /path/to/dir
```

3. 远程同步文件

SHELL

```
rsync -av /local/ user@remote:/remote/
```

4. 配置时间同步

```
vim /etc/chrony.conf  
# server ntp.example.com iburst  
systemctl restart chronyd
```

5. 应用性能配置

```
tuned-adm profile virtual-guest
```

感谢参与!

下节课见! 